

LLMsVerifier

LLMsVerifier

Проверяй. Мониторь. Оптимизируй.

Краткое описание

LLMsVerifier — это корпоративная платформа для проверки, мониторинга и оптимизации больших языковых моделей от различных провайдеров. Она основана на обязательном тесте верификации *«Видишь ли ты мой код?»*, благодаря которому только модели, доказавшие свою работоспособность, получают статус пригодных к использованию или экспорту.

Краткая характеристика

Платформа Go, которая верифицирует, бенчмаркит, мониторит и оптимизирует LLM от разных провайдеров. Каждая модель должна пройти обязательный тест на видимость кода, после чего проходит проверки на задержку, потоковую передачу, вызовы функций, обработку изображений и эмбединги, а затем экспортирует только верифицированные конфигурации для инструментов AI и CLI.

Подробное описание

LLMsVerifier — это комплексная платформа для верификации, мониторинга и оптимизации производительности LLM от различных провайдеров. Её основополагающий принцип — *обязательная верификация*, и платформа не идёт на компромиссы: прежде чем модель получит статус пригодной к использованию или будет включена в экспортируемую конфигурацию, она должна успешно пройти тест *«Видишь ли ты мой код?»*. Этот тест выполняет реальные HTTP-запросы к провайдеру и анализирует ответ на предмет подлинного понимания, а не правдоподобного эха. Модель, которая не способна продемонстрировать видимость и понимание

входных данных, просто не получает флаг «пригодна». После этого этапа движок верификации проводит полный комплекс проверок возможностей — на существование, отзывчивость, задержку, потоковую передачу, вызовы функций, обработку изображений, embeddings — а движок отчётности преобразует результаты в markdown и JSON-отчёты, на основе которых можно принимать решения.

Система модульная и событийно-ориентированная, предоставляя интерфейсы CLI, TUI, веб и REST API на базе ядра из движка верификации, движка отчётности и менеджера конфигураций. Но на верификации платформа не останавливается. Продвинутое ядро добавляет паттерн Супервизор/Воркер для декомпозиции задач на базе LLM, управление контекстом с помощью скользящего окна и LLM-суммаризации, чтобы длинные сессии не обрывались, облачное резервное копирование, а также систему аварийного переключения с автоматическими выключателями и маршрутизацией на основе задержек. Инфраструктура платформы ориентирована на промышленную эксплуатацию: шина событий pub/sub, планировщик cron, обнаружение цен и лимитов, vector-база данных для RAG и система экспорта. Фирменный брендинг добавляет суффикс (*llmsvd*) к каждому сгенерированному провайдеру/модели, благодаря чему верифицированный вывод легко отследить с первого взгляда, и его невозможно спутать с непроверенным. При этом в экспортируемые конфигурации для инструментов AI CLI, таких как OpenCode, Crush и Claude Code, попадают только верифицированные модели. Платформа поставляется с инструментарием, который действительно необходим командам в продакшене: деплой Docker/Kubernetes/Helm, мониторинг Prometheus/Grafana, LDAP/SSO и хранилище с шифрованием SQLCipher.

Зачем мы её создали

Потому что проверка только конфигурации ненадёжна — API-ключ может истечь, модель может быть устаревшей, а конфигурационный файл ничего не скажет о реальной задержке, реальных ошибках или о том, видит ли модель ваш ввод и действительно ли его понимает. LLMsVerifier заменяет принцип «*это есть в конфиге, значит, должно работать*» на доказательство: статус пригодных к использованию и экспорту получают только те модели, которые продемонстрировали корректный отклик.

Почему это меняет правила игры

LLM-флоты становятся *надёжными* — слово, которое редко заслуживают в пространстве, где конфигурации обманывают умолчанием. Вместо того чтобы надеяться, что настроенная модель сработает, команды получают гарантию, подкреплённую тестами: каждая модель проходит реальную верификацию, а мониторинг, аварийное переключение и экспорт только проверенных данных замыкают цикл от доказательства до продакшена. В экосистеме Helix это становится единым источником истины для моделей, провайдеров и метаданных верификации LLM: другие сервисы (в том числе HelixTranslate) ориентируются на него, и вся платформа получает один честный ответ на вопрос «какие модели действительно работают прямо сейчас?» вместо того, чтобы каждая команда строила собственные предположения.

Что здесь инновационного

- **Обязательная верификация «Видишь ли ты мой код?»** — реальный, подкреплённый HTTP-протоколом барьер понимания, который модель должна преодолеть, прежде чем её можно будет использовать; фирменная особенность продукта и причина, по которой ничто непроверенное не проскальзывает в систему.
- **Экспорт только проверенных конфигураций** — сгенерированные конфиги для инструментов AI CLI содержат *только* модели, прошедшие верификацию, так что в развёртываемой конфигурации не может незаметно оказаться сломанная модель.
- **Система брендированных суффиксов (llmsvd)** — каждый сгенерированный провайдер или модель несёт отслеживаемый суффикс, делая происхождение проверенных данных видимым везде, куда попадает результат.
- **Определение возможностей** для множества агентов и провайдеров CLI — система распознаёт типы потоковой передачи (SSE, WebSocket, JSONL, EventStream), сжатие и поведение кэширования, вместо того чтобы делать предположения.
- **Устойчивое аварийное переключение** — автоматические выключатели, маршрутизация по задержке (перенаправление при превышении порога времени до первого токена), проверки работоспособности и взвешенное распределение трафика поддерживают отзывчивость флотов, даже если отдельные провайдеры дают сбой.
- **Долгосрочная автономность** — паттерн декомпозиции «Супервизор/Рабочий» в сочетании с контрольными точками и интеграцией памяти позволяет поддерживать

длительные сессии, которые в противном случае исчерпали бы контекст.

- **Интеграция с RAG / vector-DB** для расширения контекста на основе достоверных данных.

Главные технические вызовы и их решения

- **Доказательство, что модель действительно работает, а не просто настроена.** В этом суть — и самая сложная часть. Решение: обязательный тест на видимость кода, который совершает реальные вызовы API и анализирует ответы на предмет подтверждённого понимания, подкреплённый широким набором тестов возможностей, а затем — отказ от экспорта всего, что не прошло проверку. Таким образом, в продакшен попадает не конфигурация, а доказательство.
- **Надёжность при работе с множеством ненадёжных сторонних провайдеров.** Решение: оркестратор аварийного переключения, который воспринимает нестабильность провайдеров как норму. Автоматические выключатели переводят провайдера в статус «деградирован» после N сбоев за M секунд, маршрутизация по задержке уводит трафик от медленных точек, периодические проверки работоспособности отслеживают восстановление, а взвешенная маршрутизация балансирует между экономичными и премиальными моделями.
- **Поддержка очень долгих автономных сессий.** Решение: паттерн декомпозиции «Супервизор/Рабочий», разбивающий большие задачи на управляемые части, периодическое сохранение контрольных точек в облачное хранилище (чтобы прогресс не терялся при прерываниях) и многоуровневое управление контекстом (скользящее окно + суммаризация LLM + RAG), чтобы модель не теряла нить, не утонув в токенах.
- **Разрастание провайдеров.** Решение: множество адаптеров Go для разных провайдеров скрыто за единым интерфейсом, а реальные конечные точки перечислены централизованно — так добавление нового провайдера остаётся локальным изменением, а не вызывает цепную реакцию в кодовой базе.

Технологический стек

- **Go** — выбран в качестве основного языка платформы благодаря поддержке конкурентности; на нём работает многопоточный движок верификации, способный

параллельно проверять множество моделей, а также сопутствующие сервисы.

- **Gin** — используется как REST API-сервер, обеспечивающий JWT-аутентификацию, ограничение частоты запросов и конечные точки WebSocket/SSE.
- **SQLite + SQLCipher** — выбраны для встроенного хранилища с шифрованием на уровне базы данных, поскольку данные верификации (ключи, результаты) чувствительны и по умолчанию должны шифроваться в состоянии покоя.
- **Redis** — используется как кэширующий слой для ускорения горячих запросов верификации и поиска метаданных.
- **RabbitMQ + Kafka** — обеспечивают событийно-ориентированную архитектуру: обмен сообщениями и потоковую передачу данных, разделяющие производителей и потребителей на платформе.
- **gRPC + Protocol Buffers** — отвечают за строго типизированную межсервисную коммуникацию и транспортировку событий между компонентами.
- **QUIC / HTTP-3 (quic-go)** — поддерживают современные транспортные протоколы (в документации репозитория отмечается ограниченная доступность HTTP/3-провайдеров — это заявленная возможность, а не универсальное утверждение).
- **JWT + LDAP/NTLM** — обеспечивают корпоративную аутентификацию, позволяя платформе интегрироваться с существующими системами идентификации (в документации заявлена поддержка SSO/SAML/OIDC).
- **Viper (конфигурация), Logrus (логирование), Brotli/compress (сжатие)** — операционная инфраструктура: гибкая настройка, структурированные логи и сжатие данных.
- **Angular** — используется для одностраничного веб-приложения, визуального интерфейса верификации и мониторинга.
- **Python + JavaScript SDK** — предоставляют клиентским командам полноценный доступ, документированный через OpenAPI/Swagger.
- **Docker, Kubernetes, Helm** — обеспечивают промышленное развёртывание с мониторингом работоспособности и автоскейлингом, позволяя флоту верификации масштабироваться как любой современный сервис.
- **Prometheus + Grafana** — отвечают за метрики и дашборды, делая состояние самой платформы столь же наблюдаемым, как и модели, за которыми она следит.
- **Testify (Go) + node -test/jsdom (веб)** — используются для многоуровневого тестирования ядра на Go и веб-интерфейса.

Статус и замечания по прозрачности

- **Статус: бета-версия.** Исходный код на Go реализует реальную HTTP-верификацию (один устаревший документ, описывающий верификацию как конфигурационную, носит аспирационный характер — авторитетен код).
- **Лицензия: не определена.** В README указана MIT, а в Dockerfile — Apache-2.0; вопрос требует решения перед публикацией.
- Количество провайдеров: в README указано «12 адаптеров», но в каталоге провайдеров их около 26 — следует считать «12+ / в процессе добавления». Многочисленные файлы со статусом «FINAL/COMPLETE» носят аспирационный характер; авторитетны код, документация и go.mod.
- Репозиторий размещён в организации vasic-digital, но функционально является доверенным слоем инфраструктуры Helix LLM-кластера.

Приоритетный уровень: Helix-первичный (LLM-инфраструктурный кластер; единый источник истины для метаданных LLM/провайдеров/верификации). Уступает по приоритету HelixTrack.